

Final Project: EEG Signal Processing

Wasam Syed (20746474) & Riad Dajani (20816768)

PSYCH 420: Introduction to Computational Neuroscience

April 12th, 2022

Dr. Britt Anderson

Abstract/Summary

This report will outline the acquisition, processing and classification of signals gathered from the occipital lobe using an Electroencephalograph (EEG). This process requires the use of Fourier Transforms as a means of signal feature extraction from the EEG and a multilayer perceptron neural network in order to extract and classify relevant features from a subject's brain under specific conditions (Palaniappan, 2005). Our report is based on R. Palaniappan's paper which outlines the design for a brain computer interface that also utilizes a multilayered perceptron neural network to classify specific mental activity across many cognitive tasks and a general description of EEG Signal Processing (Palaniappan, 2005). We gathered data from an organization that collected EEG readings from 20 subjects under two different conditions being; eyes open and eyes closed (Torkamani-Azar, Kanik, Aydin, Cetin, 2020). The tools utilized in the process concern the filtering and classification of EEG signals from the occipital sensor.

Data Acquisition

EEG Data consists of voltage readings that are usually in volts (V) and are meant to capture the potential difference occurring due to the movement of ions within neurons in the brain (Saeid Sanei, 2019). Data is acquired by placing electrodes along the scalp of the participant based on a standardized system. A common example is the International 10-20 system. However, the dataset we found utilized the International 10-10 system which differs in the placement of certain electrodes as shown in the image (Cattan et al., 2018). Electrodes can be placed in either a monopolar or bipolar fashion. Bipolar readings are made up of the potential difference between opposing electrodes and monopolar readings are potential difference readings based on an average of readings across a certain subset of electrodes (Saeid Sanei, 2019). Important considerations while acquiring data are muscular tension in the head, sweat and other

factors that could potentially add error to the readings and confound results derived from those readings. We chose to use pre-acquired data for this reason.

Data Cleaning

The data acquired from an EEG Signal has various artifacts that occur due to various elements of the process. Noise is added either through the equipment or due to the environment. Due to this, data cleaning is an essential part of the process and usually consists of many intermediate steps before we can make use of the data (Saeid Sanei & Chambers, 2009). We scaled the data and calculated descriptive statistics. We also standardized the data on a normal distribution to interpret the noise in the data and the variance. We attempted to denoise the signal using the `scipy.fft` library and functions within that library but were struggling with deriving meaningful information out of the denoised signal so opted to use the non-denoised signal to conduct further analyses.

Feature extraction: Spectral Analyses

The most important part of any signal processing method is feature extraction as that creates meaningful information out of the physical data that we gather. Spectral Analyses help us transform the data into many different forms allowing us to find useful features within them. The Fourier Transform is a foundational component in generating these spectra (Saeid Sanei & Chambers, 2009).

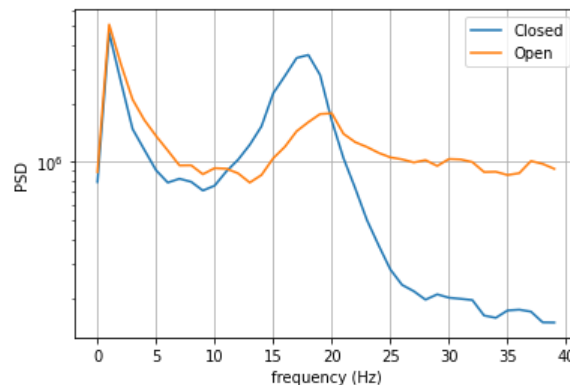
In essence, the Fourier Transform separates a mixed signal into its individual constituents made up of various frequencies, based on their spectral power (Better Explained, 2019). The formula helps find the magnitude of a given frequency by spinning the mixed signal around a circle at that desired frequency, then averaging all the various samples to get the resultant power spectral density (Australian Mathematical Sciences Institute, n.d.). This will indicate to us which

frequencies are most prevalent in the mixed signal, which is how one can differentiate and identify the various bandpowers (beta, alpha, etc.). With this, we can filter out noise and detect relevant frequencies and other features that pertain to brain activity under various conditions (Neto, 2019). The formula is as follows:

$$x[k] = \sum_{n=0}^{N-1} x[n] e^{\frac{-j2\pi kn}{N}}$$

Where $x[k]$ represents the transformed sample from the data set, N is the total number of samples in the set, k represents the frequency of the sample at the n th point in time and $x[n]$ is the value of the present sample. Because the acquisition of the signal is digital we will be using the discrete variant of the Fourier Transform formula (DFT) (Neto, 2019).

The following is a Power Spectral Density graph obtained from sampling the EEG data at 256 Hz as the data was downsampled by the researchers from 2048 Hz. Since our data was very distinct based on a simple analysis of means we did not incorporate the transform into our perceptron. However, we did use the PSD to analyze our dataset based on the Power Spectral Density for specific participants. For larger-scale applications the Fourier Transform is integrated into the feature extraction stage of EEG Signal Processing allowing for a wide range of mental states to be captured from the EEG data (Better Explained, 2019).



Classification: Multilayered Perceptron Neural Network

Finally, the features extracted from spectral analyses are inputted into machine learning algorithms to perform various functions with them. An important type of algorithm is an artificial neural network. At the surface level, artificial neural networks are rough computational models of a neuron and act in a similar manner (Saeid Sanei, 2019). Structurally, there are three types of layers that constitute the network. These are the input layer, the hidden layer(s) and the output layer. Each layer is made up of neurons that connect with every neuron in the neighboring layer (Kuldeep, 2016). These connections are called synapses and take the input from antecedent neurons, apply a weight and push the data to the next neuron. Neurons add together all the incoming outputs from adjacent synapses and apply an activation function to see if the output is ‘worthy’ of moving along the network (Saeid Sanei, 2019). These activation functions can take many forms and can be used to achieve different results. Typically, an s-curve or sigmoid function is used to determine the activation as it allows for more complex, non-linear patterns (Kuldeep, 2016).

Following, inputs and weights are formatted into matrices where they can be multiplied and summed accordingly into a new matrix that represents the activity of the current layer. Each interaction between any two layers has its own matrix of weights. As such, we have our first formula:

$$\text{layer activity} = \text{inputs} * \text{weights}$$

With the matrix representing activity, we can apply the sigmoid activation function to each entry of the activity matrix and gather a matrix of the activation values for each weighted input.

Sigmoid formula: $f(x) = \frac{1}{1 + e^{-(x)}}$, Activation formula: $activation = f(layer\ activity)$

Assuming we would like to now propagate this data into a singular output, we can repeat the first step by using the activation matrix as our input, and multiply that by our second set of weights (Kuldeep, 2016). Finally, the output neuron must apply the activation function once again, providing us with estimates of the input's classifications, denoted by $\hat{y}$. At this point, the estimates will be incorrect as a cost function will be required to account for the degree of incorrectness each estimate has. The formula is as follows:

$$C = \frac{1}{2n} \sum_x \|y(x) - a^L(x)\|^2$$

Where the original classification is subtracted by the last activation (a^L or $\hat{y}$), divided by 2, squared and summed across all estimates. This will result in the cost or error which we want to decrease. This is where we 'train our network' or reduce the cost (Saeid Sanei, 2019). We can achieve this by manipulating variables that we can directly control and have an effect on the outcome. These are the weights in our network. Conceptually, there exists some combination of weights that provides the lowest cost, however, brute forcing this combination becomes exponentially more demanding as more layers are added (Kuldeep, 2016). A technique known as 'Gradient Descent' can be used by assessing how much the cost is changed with a change in

weights. If this partial derivative is negative as a result of the weight change, then we are reducing error and should continue to change the weight in a similar direction. This can be iteratively applied and terminated once the costs cease to reduce (Kuldeep, 2016). Because we have two sets of weights, we must compute their relative costs individually, producing matrices that are of the same dimensions. Here we can utilize the chain rule, which multiplies the derivative of an outer function by the inner function and provides the derivative of a function inside of another function (Kuldeep, 2016). This can be expressed through the product of derivatives rule:

$$\frac{d(uv)}{dx} = u \frac{dv}{dx} + v \frac{du}{dx}$$

Where the subtraction of the estimate is multiplied by its derivative with respect to the change in a given weight. We can find this derivative through deriving the sigmoid activation function with respect to the layer activity (Saeid Sanei, 2019). Interestingly, this derivative will result in an activation value due to the linear relationship between weights and layer activity. In essence, this will backpropagate the error back to the activation layer and do so in a relative manner; assigning higher or lower rates of error for each weight.

Finally, we need to propagate this error to weights deeper in the network by finding the derivative across the synapses (Kuldeep, 2016). This is done by finding the derivative of the layer activity with respect to the previous neurons' activation, which will provide us with the weight value of the given synapse. Mathematically, this can be done by multiplying the derivative of each interaction with one another. Effectively, this reverses the process of

feedforward network but only in accordance to the rates of change, or the cost (Saeid Sanei, 2019).

Application

There are many use cases for EEG Signal Processing especially within the field of diagnostics and medicine. The paper our report is based on explores Brain-Computer Interfaces which are applications that open the door to human-computer interaction without the aid of physical implements (Palaniappan, 2005).

Development Tools

There are plenty of tools and programming languages that can achieve the task of EEG Signal Processing. Since we were both working with Python we chose to use tools offered by the community of Python developers to aid in our implementation. PyTorch and Scikitlearn are two robust tools used in developing machine learning applications. PySimpleGUI is a tool used to design User Interfaces using Python.

Tools Used

We used the Python “statistics”, “matplotlib” and “random” libraries to perform the math within our code implementation. Python “random” was used to generate weights for the perceptron and “statistics” was used to conduct basic descriptive statistics on the signal. “Matplotlib” was used to plot spectra. Python scipy.io was used to input the dataset that was in “.mat” (MATLAB) format and the relevant arrays were extracted with basic Python functions.

“Scipy.fft” was used to create discrete Fourier Transforms of the signal for denoising and “scipy.signal.welch” was used to create the Power Spectral Density diagram for feature analysis. We used Python “mouse” on the application side to be able to control the mouse on the screen based on the output from the perceptron.

Conclusion

Our report shows how powerful a simple implementation of EEG Signal Processing can be. We used a dataset of EEG recordings and were able to create a version of a Brain-Computer Interface. At a higher scale this methodology has many possibilities.

References

- Australian Mathematical Sciences Institute. (n.d.). *Content - Graphing the trigonometric functions*. Amsi.org.au.
https://amsi.org.au/ESA_Senior_Years/SeniorTopic2/2d/2d_2content_6.html
- Better Explained. (2019). *An Interactive Guide To The Fourier Transform – BetterExplained*.
Betterexplained.com.
<https://betterexplained.com/articles/an-interactive-guide-to-the-fourier-transform/>
- Cattan, G., Rodrigues, P. L. C., & Congedo, M. (2018, December 17). *EEG Alpha Waves dataset*.
Zenodo; Zenodo. <https://zenodo.org/record/2348892>
- Neto, J. (2019). *Fourier Transform: A R Tutorial*. Fc.ul.pt.
<http://www.di.fc.ul.pt/~jpn/r/fourier/fourier.html>
- Palaniappan, R. (2005). Brain Computer Interface Design Using Band Powers Extracted During Mental Tasks. *Conference Proceedings. 2nd International IEEE EMBS Conference on Neural Engineering, 2005.*, 0-7803-8710-4(8521288).
<https://doi.org/10.1109/cne.2005.1419622>
- Saeid Sanei. (2019). *Eeg signal processing and machine learning second edition*. Wiley.
- Saeid Sanei, & Chambers, J. A. (2009). *EEG signal processing*. John Wiley.
- Shiruru, Kuldeep. (2016). AN INTRODUCTION TO ARTIFICIAL NEURAL NETWORK.
International Journal of Advance Research and Innovative Ideas in Education. 1. 27-30.
- Thatcher, R. W. (2012). Coherence, Phase Differences, Phase Shift, and Phase Lock in EEG/ERP Analyses. *Developmental Neuropsychology*, 37(6), 476–496.
<https://doi.org/10.1080/87565641.2011.619241>

Torkamani-Azar, M., Kanik, S. D., Aydin, S., & Cetin, M. (2020). Prediction of Reaction Time and Vigilance Variability From Spatio-Spectral Features of Resting-State EEG in a Long Sustained Attention Task. *IEEE Journal of Biomedical and Health Informatics*, 24(9), 2550–2558. <https://doi.org/10.1109/jbhi.2020.2980056>